

Lecture 10

RISC-V: Pipelined Implementation – Control Hazards

Instructor: Dr. Muhammad Imran



Computer Architecture | RISC-V Pipelined Implementation

Acknowledgement

- Content from following has been used in these lectures
 - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy

Contents

- Control Hazards
- Branch Prediction
 - Static Prediction
 - Dynamic Prediction

Addressing Control Hazards in Pipeline

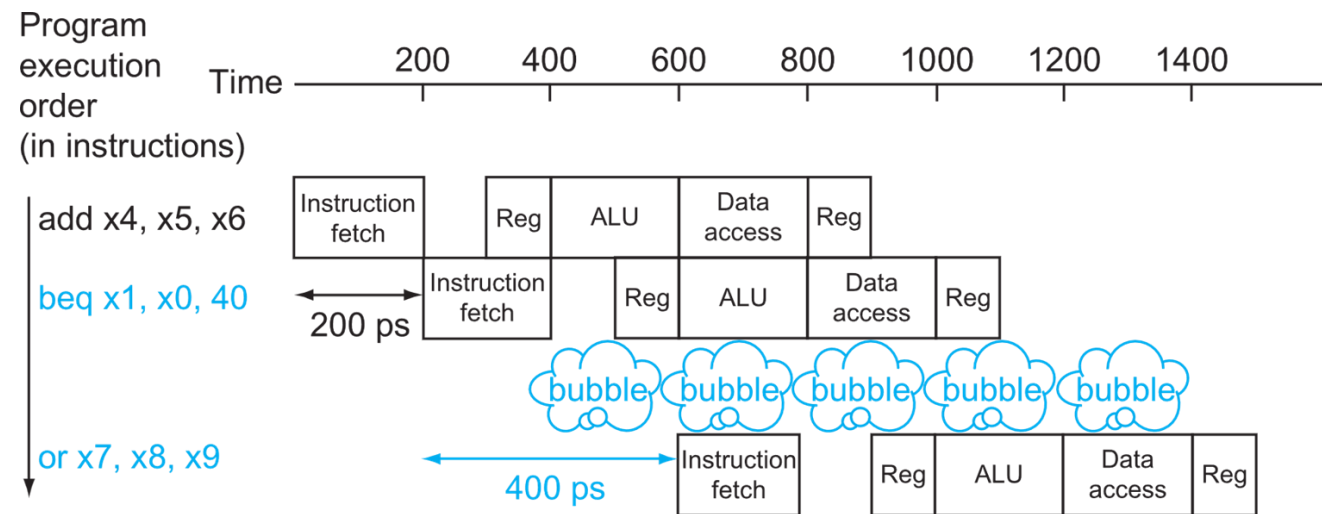


Control Hazards

- When conditional branch instructions need to decide which instruction should be next
 - Next instruction cannot be **fetch**ed until branch decision is finalized!
- Also known as branch hazards
- Simple **solu**tion?
 - Stall the pipeline until branch is decided!
 - Implementation is same as in case of data hazards, need to enhance hazard detection logic to detect control hazards!
 - How many stalls?
 - If branch is decided in 3rd stage and PC is updated in 4th stage!
 - Three stalls!
 - If branch is decided in 3rd stage and PC is also updated in 3rd stage!
 - Two stalls!

Reducing Branch Delay

- Branch decision can be finalized in ID stage by adding extra hardware (xor gates for comparison + adder for PC+offset)!
 - Still the pipeline may need to be stalled!
 - Only one stall as opposed to 3!
 - Example



Reducing Branch Delay

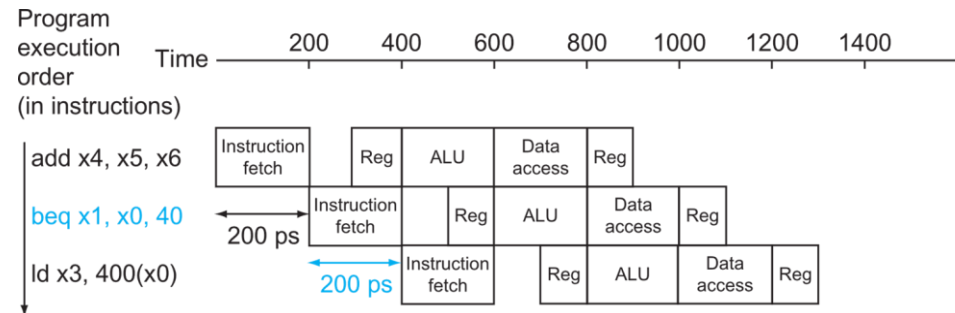
- Branch decision can be finalized in ID stage by adding extra hardware (xor gates for comparison)!
 - Problems?
 - Forwarding (for RAW dependency) needs to be modified!
 - If branch is dependent on an earlier instructions, forwarding may be needed to ID stage where branch is executing!
 - Forwarding sources can be EX/MEM or MEM/WB registers!
 - Additional stalls (for RAW dependency) may also be needed!
 - If branch is dependent on ALU instruction → 1 Cycle Stall
 - If branch is dependent on previous load → 2 Cycle Stall

Addressing Control Hazards

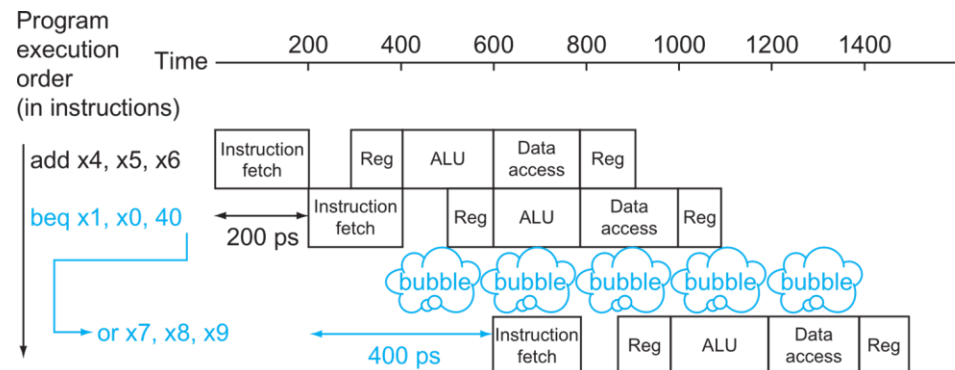
- Prediction
 - A better solution to control hazards
 - Predict the outcome of branch and fetch the next instruction!
 - If prediction is wrong, fetch the right instruction again!
- Prediction can be static or dynamic!

Addressing Control Hazards

- Static prediction
 - Predict **all branches** as **taken** or **not taken**
 - Last example using prediction
 - When prediction is correct!



- When prediction is wrong!



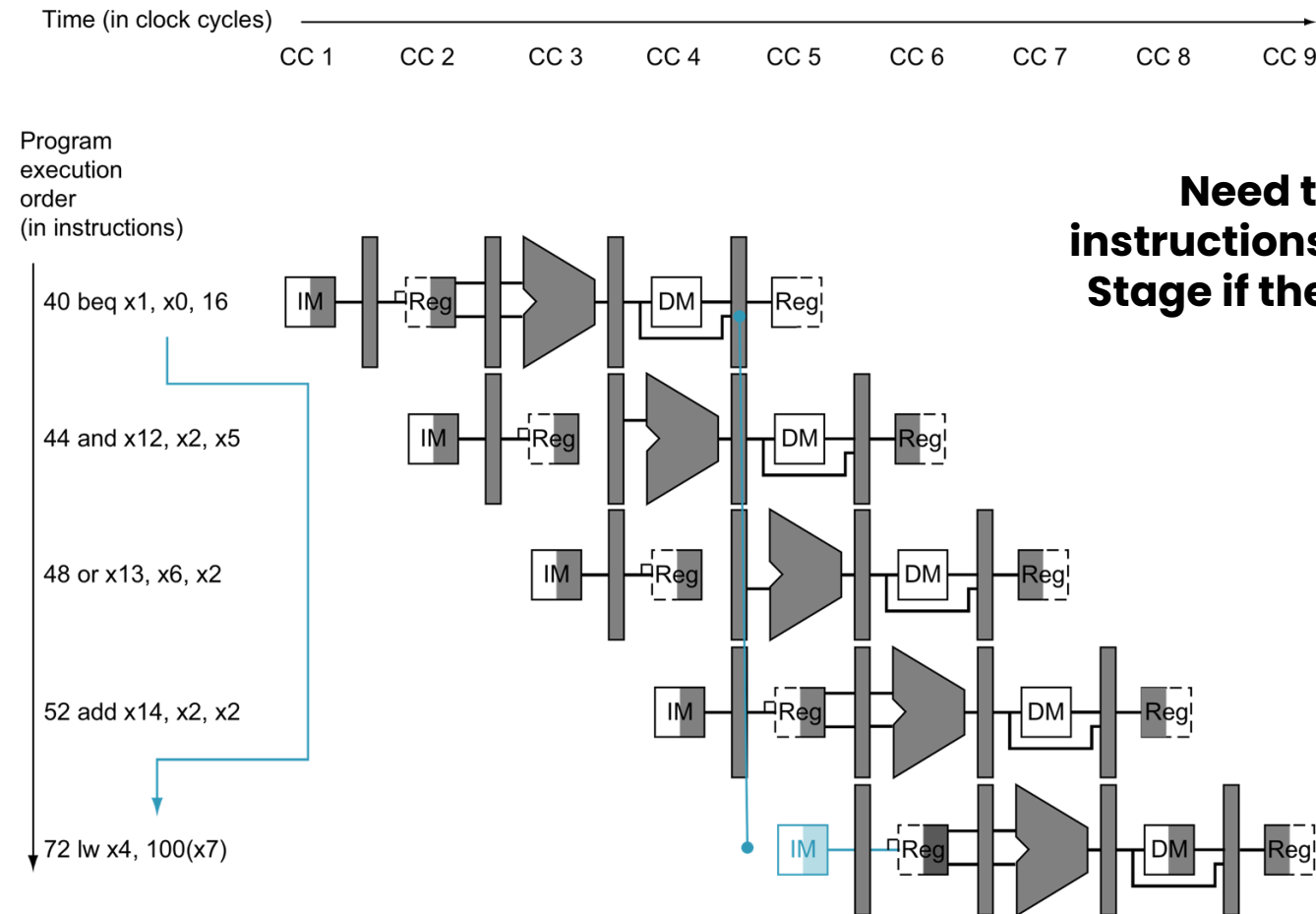
Addressing Control Hazards

- Static prediction
 - Predict **some branches** as **taken** and some as **not taken**
 - For example, a branch instruction at the end of a loop is usually taken
 - Predict all branches to earlier addresses to be taken!
- When predicting '**taken**'
 - Pipeline may still be stalled until branch target address (PC+offset) is available!
 - Use target address buffer to avoid these stalls!
 - Example
 - If address is computed in ID stage and branch is also decided in ID stage!
 - One stall when predicting as "taken"!
 - No stall when predicting as "not taken"!

When prediction goes wrong ... some instruction in the pipeline may need to be discarded!

Addressing Control Hazards

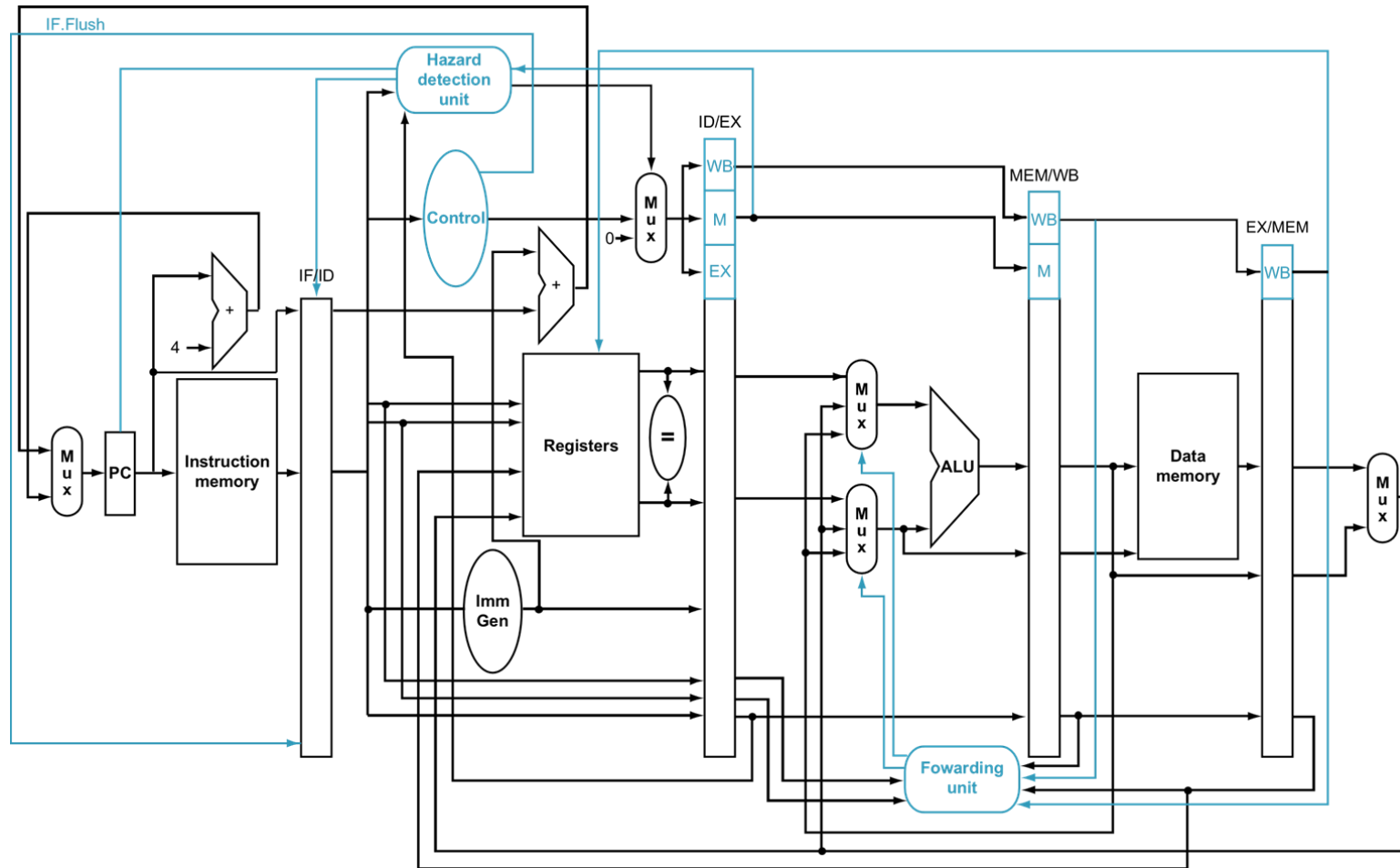
- Assuming branch decision comes after MEM Stage!



Discarding Instructions

- Instruction entering MEM stage
 - Make control signals as zeroes!
- Instruction entering EXE stage
 - Make control signals as zeroes!
- How to discard instruction in IF/ID register which is fetched in same cycle when branch is decided!
 - Will enter ID stage in next cycle!
 - The instruction is not yet decoded and there are no control signals to be zeroed!
 - How to handle this?
 - Use a control signal **IF.Flush** which zeroes the instruction in IF/ID register!
 - The instruction effectively becomes *nop instruction*!

Final Pipelined Datapath and Control



- Assuming branches are implemented in ID stage!
- Note: ALU Source Mux + Mux control signals not shown ...

Addressing Control Hazards

- Delayed prediction
 - Delay the branch decision!
 - Execute the instruction which is not impacted by branch!
 - Efficient for one-cycle branch delays!
 - Example
 - add x1, x2, x4
 - beq x5, x6, somewhere
 - Reorder the instructions as
 - beq x5, x6, somewhere
 - add x1, x2, x4
 - Can be handled by assembler!
 - Invisible to the programmer!

Addressing Control Hazards

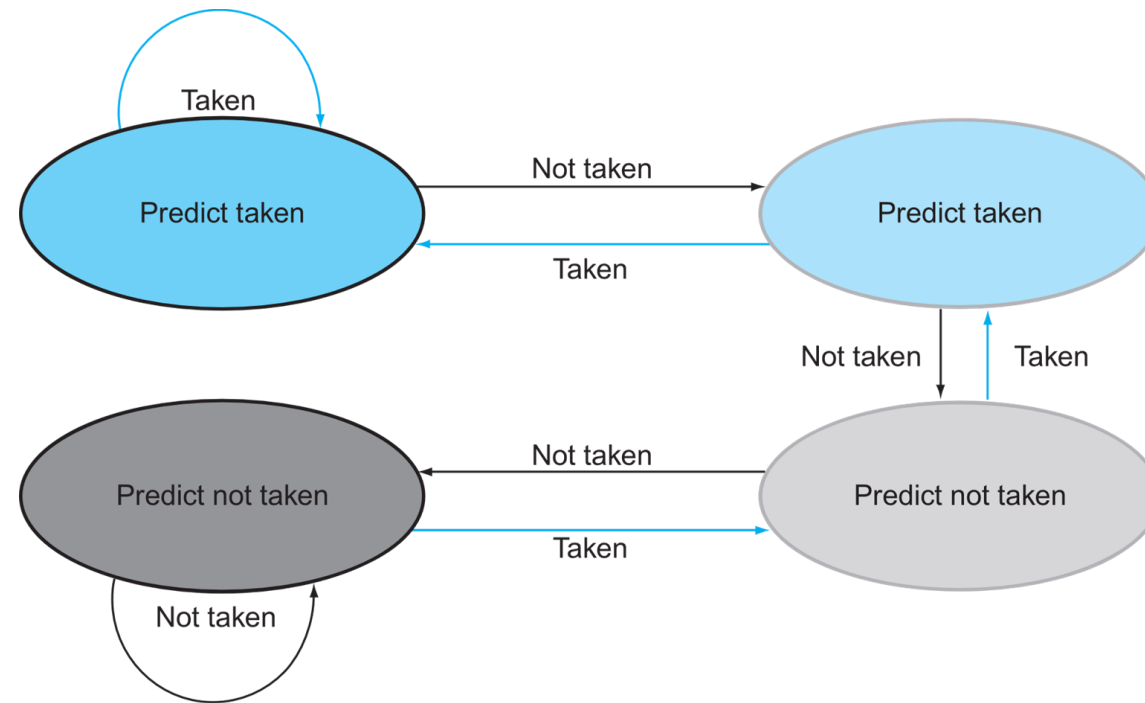
- Dynamic prediction
 - Predict based on knowledge of the behavior of branch instruction!
 - Keep history of different branches as taken and not taken
 - Predict based on prevalent behavior of different instructions!
 - Because of lot of history, such prediction have accuracy of above 90%!

Dynamic Branch Prediction

- To implement dynamic prediction in hardware
 - Include a small look-up table which is indexed by the lower portion of the address of branches
 - Branch Prediction Buffer
 - Branch History Table
 - A single bit may indicate if this branch was taken or not taken recently!
 - Prediction bit is inverted on misprediction!
 - Accuracy of such predictor in loops?
 - 1 misprediction on start of loop!
 - $(n-1)$ correct predictions!
 - 1 misprediction when exiting loop!
 - Success rate = $(n-1)/(n+1)$, e.g. for 9 iterations, it is 80%!
 - Can we improve accuracy?

Dynamic Branch Prediction

- 2-bit branch prediction!
 - Revert branch decision on two mispredictions not one!



Dynamic Branch Prediction

- Tournament Branch Predictor
 - Use multiple branch predictors!
 - A selector selects the predictor which has been most accurate for a given branch!
- Target Address Calculation
 - For taken branches target address calculation requires 1 cycle!
 - To avoid 1 cycle penalty, use
 - Branch Target Buffer
 - A cache to hold target PC or instruction for branches!

Knowledge Check!

- Consider three branch prediction schemes: predict not taken, predict taken, and dynamic prediction. Assume that they all have zero penalty when they predict correctly and two cycles when they are wrong. Assume that the average predict accuracy of the dynamic predictor is 90%. Which predictor is the best choice for the following branches?
 1. A conditional branch that is taken with 5% frequency
 - Answer
 - Predict Not taken!
 2. A conditional branch that is taken with 95% frequency
 - Answer
 - Predict Taken!
 3. A conditional branch that is taken with 70% frequency
 - Answer
 - Dynamic Prediction is better!

**Instruction Sets can make pipelining
easier or harder ...**

Thank You

